STRATÉGIE DE TEST

Preuve de concept - orientation hospitalière
en situation d'urgence

Ce document définit la stratégie de test du **PoC MedHead**, les environnements mobilisés, les jeux de données, les scénarios de validation, les résultats mesurés et les limites techniques identifiées.

1 290	7	100 000
hôpitaux de test	tests backend validés	requêtes JMeter

Objectif : vérifier les fonctions de recherche d'établissements, la cohérence des interfaces, la reproductibilité des tests et les performances du backend sur un périmètre défini.

Composants évalués : API Spring Boot, base H2, interface Vue.js, exécution conteneurisée, intégration continue et service de routage externe.

Dépôt du code : https://github.com/naegel23/P11-MedHead-PoC

Dépôt d'architecture : https://github.com/naegel23/P11-MedHead-Architecture

Portée : preuve de concept sur données de test. Les résultats ne constituent ni une qualification médicale ni une homologation de production.

Sommaire et périmètre

01 Introduction et pyramide des tests

02 Environnement technique

03 Jeu de données de test et provenance

04 Bibliothèques et outillage

05 Tests unitaires

06 Tests d'intégration et d'API

07 Tests fonctionnels et parcours de bout en bout

08 Tests de performance et critères mesurés

09 Intégration continue et rapports de tests

10 Limites, risques et prochaines validations

Méthode de lecture

Chaque contrôle décrit **le comportement recherché**, **la méthode de test** et, lorsque cela est disponible, **la preuve observée**. Les vérifications réalisées sont séparées des exigences restant à couvrir, afin de permettre un suivi précis des critères d'acceptation.

Statut	Signification
Validé sur le périmètre testé	Le résultat a été constaté dans des conditions précisées. Il n'est pas généralisé à la production.
Scénario à couvrir	Cas identifié dans le plan de test ; une preuve d'exécution reste nécessaire.
Hors du périmètre	Exigence de la solution cible ne pouvant pas être évaluée avec ce seul PoC.

01 Introduction et pyramide des tests

Le PoC MedHead expérimente une capacité d'orientation vers un établissement hospitalier selon la position géographique de l'urgence, la spécialité médicale demandée et le nombre de lits simulés disponibles. La stratégie de test vérifie à la fois les comportements applicatifs et les risques techniques clés : latence, intégration, reproductibilité et dépendance au service de routage.

La **pyramide des tests** distingue trois niveaux : tests unitaires à la base, tests d'intégration au centre et tests fonctionnels / parcours de bout en bout au sommet. Des tests de performance transversaux complètent ces niveaux.

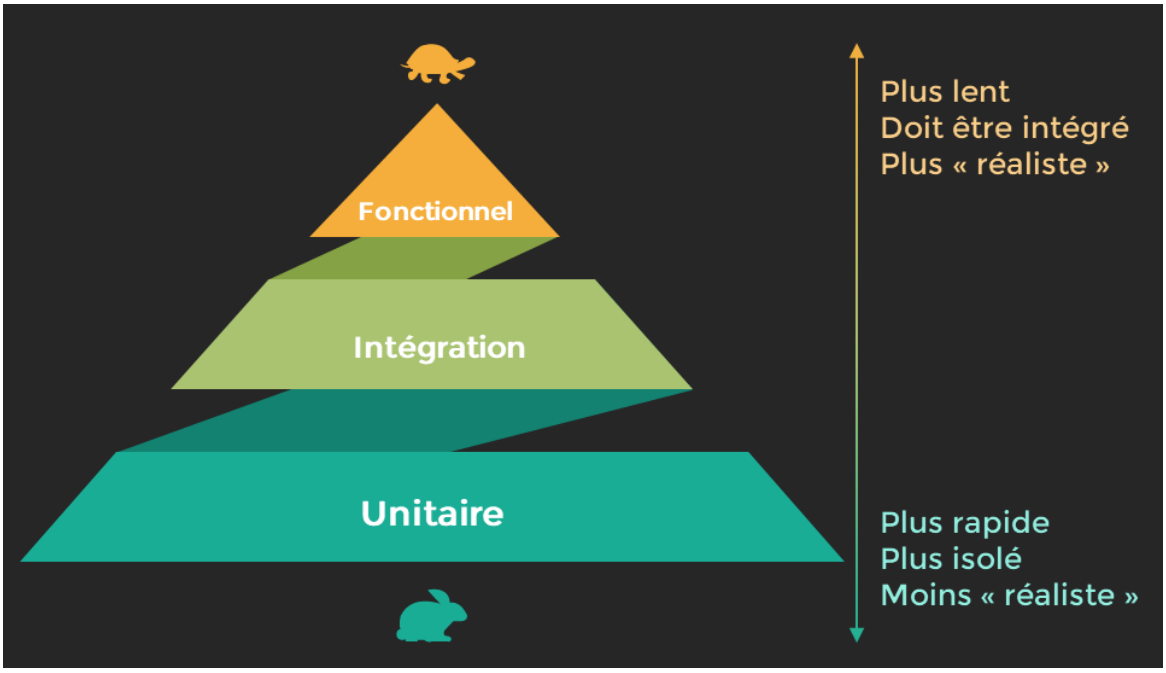


Figure 1 — Pyramide des tests : couverture unitaire, intégration, validation fonctionnelle et contrôle de performance.

Niveau	Finalité	Mode de validation
Unitaires	Tester une logique ou une méthode isolée.	JUnit 5, éventuellement avec Mockito.
Intégration / API	Tester l'assemblage des couches et le contrat HTTP.	Exécution des tests backend et contrôles HTTP.
Fonctionnels / E2E	Vérifier le parcours utilisateur visible.	Tests frontend et parcours manuel Vue → API.
Non fonctionnels	Vérifier débit, latence et taux d'erreur.	Apache JMeter, conditions reproductibles.

02 Environnement technique

Le PoC est constitué d'une API Java, d'une interface Vue.js, d'un jeu de données H2 et d'une chaîne d'intégration continue. Les principaux composants et outils utilisés sont précisés ci-dessous.

Composant	Configuration utilisée	Rôle
Backend	Java 17 ; Spring Boot 3.5.16	API REST et logique d'orientation.
Persistance	Spring Data JPA ; Hibernate 6 ; H2 2.3.232	Base de données embarquée et jeu de test.
Documentation API	OpenAPI 3 ; springdoc 2.8.17	Contrat de service et interface Swagger UI.
Frontend	Vue 3.5 ; Vite 8 ; Node.js 22 pour la CI	Saisie de recherche et rendu des résultats.
Tests backend	JUnit 5 ; Maven ; profil Failsafe	Validation automatisée.
Tests frontend	Vitest ; Vue Test Utils	Tests de composants et de comportement.
Charge	Apache JMeter	Mesures de performances locales.
CI	GitHub Actions ; Ubuntu 24.04	Tests, compilations et artefacts.
Conteneur	Docker ; runtime Eclipse Temurin 17 JRE	Vérification de portabilité du backend.

Interfaces de validation locale

- API Spring Boot : **http://localhost:9000** (lorsqu'elle est démarrée localement).
- Spécification OpenAPI : **http://localhost:9000/v3/api-docs** — HTTP 200 observé.
- Swagger UI : **http://localhost:9000/swagger-ui/index.html** — HTTP 200 observé.
- Interface Vue : session locale testée sur **http://localhost:5174** ; le port peut varier selon Vite.

Le test d'intégration interface/API a exploité la configuration de proxy Vite **/api → http://localhost:9000**. Ces adresses sont des références de développement et ne constituent pas des URL de production.

03 Jeu de données : provenance et limites

Origine et préparation

Le jeu d'essai est constitué à partir de données d'établissements hospitaliers attribuées au **National Health Service (NHS)**. La source CSV renseignée pour cette préparation est : https://assets.nhs.uk/data/foi/Hospital.csv. La disponibilité et l'intégrité du CSV à cette adresse ne sont pas vérifiées par les campagnes de test présentées ici.

Les colonnes des établissements incluent notamment nom, adresse et coordonnées géographiques. Les **spécialités médicales sont affectées de façon simulée** à partir d'une nomenclature NHS et la valeur **freebed** est générée entre **0 et 150**. Ces attributs ne reflètent ni des effectifs hospitaliers ni des lits disponibles en temps réel.

OrganisationName	Longitude	Latitude	specialities	address	freebed
Walton Community Hospital - Virgin Care Services Ltd	-0.40604206919670105	51.379997253417969	oral surgery,oral surgery,general practice (gp) 6 month training,endodontic	Rodney Road Walton-on-Thames Surrey KT12 3LD	118
Woking Community Hospital (Virgin Care)	-0.55628949403762817	51.315132141113281	oral and maxillo-facial surgery,genito-urinary medicine,dermatology,intens	Heathside Road Woking Surrey GU22 7HS	89
North Somerset Community Hospital	-2.8471927642822266	51.43719482421875	restorative dentistry,restorative dentistry,paediatric dentistry,audio vestib	North Somerset Community Hospital Old Street Clevedon Avo	83
Bridgewater Hospital	-2.2454688549041748	53.459743499755859	clinical genetics,restorative dentistry,clinical pharmacology and therapeut	120 Princess Road Manchester Greater Manchester M15 5AT	0
Kneesworth House	-3.0604055151343346E-2	52.078121185302734	audio vestibular medicine,dermatology,oral medicine,intensive care medic	Old North Road Bassingbourn Royston SG8 5JP	0
Stockton Hall Hospital	-1.0025526285171509	53.995403289794922	dermatology,dental and maxillofacial radiology,general med practitioner,re	Stockton Hall The Village Stockton On The Forest York North	0
Lakeside Hospital	-0.31203153729438782	52.198459625244141	oral and maxillofacial pathology,periodontics,clinical genetics,prosthodont	The Lane Wyboston Bedford Bedfordshire MK44 3AS	140
Orthopaedics and Spine Specialist Hospital	-0.28302180767059326	52.604942321777344	special care dentistry,periodontics,clinical oncology,clinical genetics,oral	1 Stirling Way Bretton Peterborough Cambridgeshire PE3 8YA	0
Lee Mill Hospital	-3.9596600532531738	50.389484405517578	oral medicine,additional dental specialties,special care dentistry,dermatol	Beech Road Ivybridge Plymouth Devon PL21 9HL	0
Mount Gould Hospital	-4.1127405166625977	50.378402709960938	general practice (gp) 6 month training,emergency medicine,acute internal	Mount Gould Road Plymouth PL4 7QD	137
Nuffield Health	-1.8697808980941772	50.729755401611328	prosthodontics,emergency medicine,audio vestibular medicine,clinical ne	Bournemouth Hospital 67 Landsdowne Road Bournemouth Do	76
Nuffield Health	0.31283530592918396	51.623332977294922	intensive care medicine,oral and maxillofacial pathology,endodontics,card	Brentwood Hospital Shenfield Road Brentwood Essex CM15 8	31

Figure 2 — Exemple des champs du jeu d'essai : établissement, géolocalisation, spécialités et lits simulés.

Chargement et reproductibilité

Les données sont définies dans le fichier **data.sql** et chargées au démarrage de l'application dans une base embarquée **H2**. Le jeu utilisé compte **1 290 établissements** et permet de reproduire les filtres par spécialité, disponibilité et rayon de recherche.

Champ	Usage dans les tests
hospital_name, address	Identifier l'établissement figurant dans la réponse API.
longitude, latitude	Calculer l'éligibilité à un rayon de recherche.
spec	Tester la correspondance des spécialités simulées.
freebed	Tester la condition de disponibilité fictive (0 à 150).

Limite : ce dataset permet des tests logiciels reproductibles, mais pas de mesurer la fiabilité réelle de la gestion des lits, ni de valider des décisions de prise en charge des patients.

04 Bibliothèques et outillage

JUnit 5

Cadre d'exécution des tests Java automatisés. Les assertions permettent de contrôler les réponses et le comportement des composants.

Mockito

Création de doubles de tests pour isoler les dépendances et contrôler les interactions dans les scénarios unitaires concernés.

Spring Boot Test / Maven

Démarrage du contexte de test lorsque nécessaire, orchestration des suites avec Maven et intégration à la CI.

Vitest + Vue Test Utils

Vérification des composants Vue et de la recherche à l'aide de réponses backend simulées.

Apache JMeter

Génération de charge, collecte des latences, du taux d'erreur et des percentiles pour un endpoint donné.

OpenAPI 3 / Swagger UI

Contrat HTTP découvrable et interface de consultation des routes exposées. Swagger UI ne constitue pas, à lui seul, un test fonctionnel automatisé.

Couverture des tests et analyse statique

La couverture de code peut être mesurée avec **JaCoCo** lorsque son exécution est configurée dans la chaîne Maven. Un indicateur exploitable doit inclure le taux de couverture, les classes concernées et les cas non couverts. Aucun taux chiffré de couverture n'est avancé ici sans rapport d'exécution correspondant.

L'analyse statique complète cette vérification en signalant les vulnérabilités potentielles, les duplications et les problèmes de maintenabilité. Elle doit être exécutée avec un outil activé dans le projet avant qu'un résultat soit revendiqué. Les répertoires Maven usuels des rapports sont **target/surefire-reports** et **target/failsafe-reports**.

05 Tests unitaires

Les tests unitaires vérifient la logique métier sur des jeux de données contrôlés, sans solliciter le routage externe. Ils doivent isoler les cas limites : absence de lits disponibles, spécialité inconnue, rayon invalide, établissement absent et collections vides.

Cas à couvrir dans la couche repository

Méthode ou opération	Comportement à vérifier
findByFreebed()	Ne retourner que les établissements ayant au moins un lit simulé disponible.
findBySpecialities()	Filtrer les établissements selon la spécialité demandée.
findByFreebedAndBySpecialities()	Combiner les critères de lit libre et de spécialité.

Cas à couvrir dans la couche service

Opération	Comportement à vérifier
getAllHospital()	Retourner une collection cohérente avec les données injectées.
getHospital()	Retourner le bon établissement ou gérer l'absence d'identifiant.
findByFreebed()	Écarter les établissements sans lit simulé disponible.
getAllHospitalInRange()	Ne retenir que les établissements dans le rayon demandé.

Résultat automatisé constaté

La validation du backend a permis l'exécution réussie de **7 tests automatisés**. Ce total décrit la suite exécutée ; il ne doit pas être interprété comme une preuve que tous les scénarios listés ci-dessus sont couverts individuellement. La matrice de couverture doit être rapprochée des classes et rapports de tests Maven.

06 Tests d'intégration et d'API

Les tests d'intégration vérifient la collaboration entre les couches **contrôleur, service et repository**, ainsi que les codes HTTP et les objets JSON retournés. Les scénarios ci-dessous définissent les principales vérifications du contrat d'API.

Cas d'API	Attendu / vérification
GET /hospital	Liste des hôpitaux du jeu SQL ; 1 290 en configuration de référence.
GET /hospital/4	Établissement identifié par 4 ; données cohérentes du jeu SQL.
GET /hospital/speciality/pharmacology	Liste filtrée selon la spécialité fournie.
GET /hospital/free/speciality/pharmacology	Présence de lits simulés libres et spécialité demandée.
GET /hospital/400258	Erreur contrôlée si l'identifiant est introuvable.
GET /hospital/free/speciality/pharmacology/range/{lon}/{lat}/{rayon}	Recherche combinant lit, spécialité et rayon. Route utilisée pour le benchmark.
GET /nearest/{lon}/{lat}/{rayon}/{spec}	Recommandation avec service de routage externe ; cas soumis aux quotas fournisseur.

Contrôles de contrat API

- Vérifier codes HTTP et structure JSON ; différencier les cas nominaux, vides et invalides.
- Vérifier la cohérence du filtrage par spécialité, disponibilité et rayon sur le dataset fixe.
- Documenter les paramètres via **/v3/api-docs** et l'interface **/swagger-ui/index.html**.
- Distinguer les réponses issues du backend local de celles dépendant du fournisseur OpenRouteService.

Résultats observés : démarrage de l'API, accès aux interfaces OpenAPI et Swagger UI avec réponse HTTP 200, et exécution du backend dans un conteneur Docker avec une requête API retournant HTTP 200.

07 Tests fonctionnels et parcours E2E

Les vérifications fonctionnelles portent sur le parcours de recherche proposé par l'interface Vue.js : saisie des paramètres, communication avec l'API REST, traitement de la réponse et présentation des établissements. Un parcours complet a été exercé manuellement en environnement local.



Le service de routage OpenRouteService est une dépendance externe du parcours /nearest.

Scénario manuel de bout en bout observé

Étape	Paramétrage ou contrôle
1. Ouvrir l'interface	Vue.js servie localement sur localhost:5174.
2. Saisir la recherche	Spécialité : pharmacology ; longitude : -2.0115245 ; latitude : 52.00514984 ; rayon : 100 km.
3. Lancer la requête	L'interface contacte le backend sur /api via le proxy de développement Vite.
4. Vérifier le résultat	Retour HTTP 200 et 20 établissements présentés dans le scénario local contrôlé.
5. Examiner l'affichage	Les informations hospitalières et les résultats d'itinéraire s'affichent dans l'interface.

Tests automatiques de l'interface

La suite frontend repose sur **Vitest** et **Vue Test Utils** : affichage du formulaire et déclenchement d'une recherche avec réponse du backend simulée. Ces tests s'exécutent dans la CI, mais **ne se substituent pas à une véritable suite E2E automatisée** avec navigateur et backend en fonctionnement.

Scénarios encore à étendre

- Aucun résultat ; spécialité inconnue ; entrées géographiques invalides.
- Perte de réseau, indisponibilité de l'API de routage, code 429 et message utilisateur.
- Automatisation E2E du parcours navigateur avec vérifications stables sur un jeu de référence.

08 Tests de performance

La campagne de performance utilise **Apache JMeter** sur un endpoint backend local de recherche par spécialité, disponibilité et rayon. Elle ne sollicite **pas** le calcul d'itinéraire OpenRouteService. Elle valide donc un **périmètre technique circonscrit**, non l'ensemble de la chaîne métier.

Protocole de test

Paramètre	Valeur observée ou configurée
Endpoint	/hospital/free/speciality/pharmacology/range/-2.0115245/52.00514984/100
Population de test	100 utilisateurs virtuels ; 1 000 boucles chacun.
Nombre de requêtes	100 000.
Objectif de débit	800 requêtes par seconde.
Débit réellement mesuré	998,93 requêtes par seconde.
Taux d'erreur	0 % pendant cette exécution.
Environnement	Backend local, sans dépendance de routage externe.

Latences mesurées

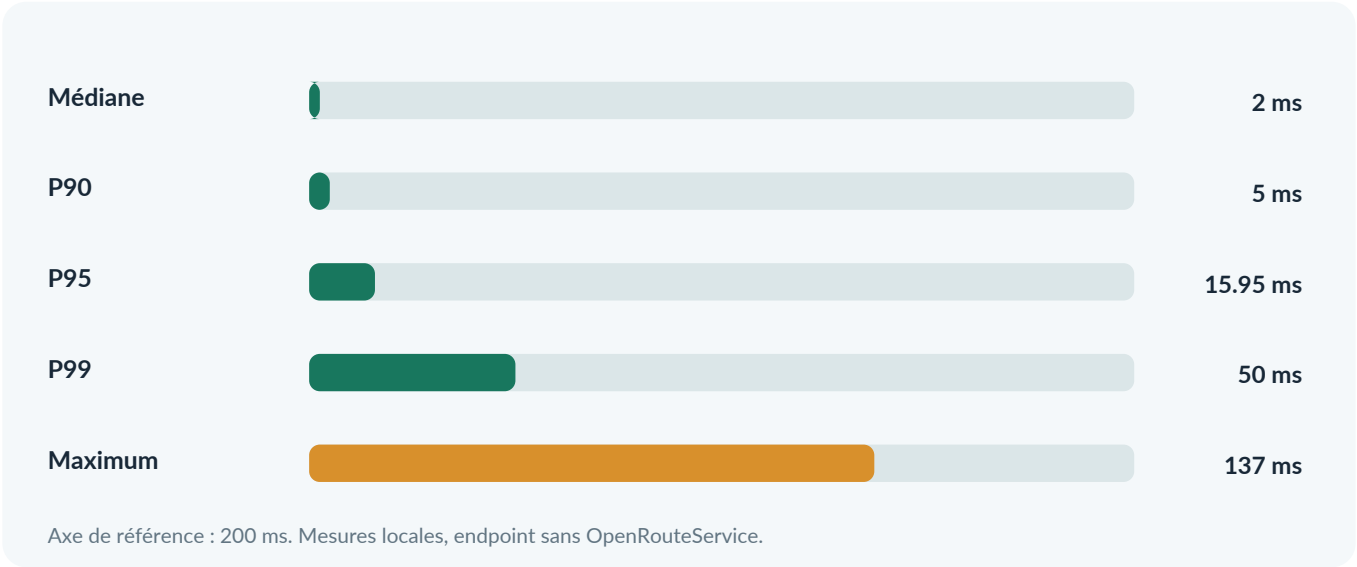


Figure 3 — Percentiles et maximum de l'exécution JMeter. Tous les chiffres se rapportent à ce seul scénario local.

La latence **moyenne** est de **3,98 ms**. La médiane est de 2 ms, le P95 de 15,95 ms, le P99 de 50 ms et le maximum de 137 ms. Le débit mesuré est supérieur à l'objectif de 800 req/s et les latences observées restent sous 200 ms lors de cette exécution.

08 Interprétation des résultats

Ce que le benchmark démontre

- Sur l'endpoint testé, **100 000 requêtes** sont passées avec **0 % d'erreur** lors du run retenu.
- Le débit effectivement mesuré de **998,93 req/s** dépasse le débit cible de 800 req/s dans ce contexte.
- Le **maximum observé de 137 ms** reste inférieur au seuil cible de 200 ms pour le scénario exécuté.

Ce que le benchmark ne démontre pas

- Une latence inférieure à 200 ms pour la route **/nearest** utilisant OpenRouteService.
- Une disponibilité, une tolérance aux pannes ou un débit équivalents en environnement de production.
- L'atteinte du KPI métier **plus de 90 %** d'orientation vers le bon établissement.
- Le passage du délai moyen de traitement d'une urgence de **18,25 à 12 minutes**.
- La résilience face à une indisponibilité du fournisseur de routage ou une panne de l'infrastructure.

Dépendance externe observée

Les essais impliquant **OpenRouteService** ont rencontré des réponses HTTP **429 Too Many Requests** dues aux quotas du fournisseur. Dans certains cas, cette limitation peut entraîner une réponse **408 Request Timeout** côté backend. Les résultats JMeter présentés ici **excluent expressément** ce chemin externe.

Risque	Mesure à envisager avant production
Quota et indisponibilité fournisseur	Accord de service adapté, supervision des quotas, limites de concurrence.
Dégradation des temps de réponse	Timeouts explicites, retries bornés avec backoff, cache si acceptable.
Dépendance unique	Stratégie de repli et tests de défaillance supervisés.

09 Automatisation CI et rapports de tests

La chaîne d'intégration continue GitHub Actions s'exécute lors des **push** et **pull requests** ciblant la branche **main**. Elle vérifie les composants backend et frontend, exécute les tests et construit les artefacts. Les vérifications du workflow ont été exécutées avec succès.

Chaîne de contrôle	Preuve disponible / précaution
Backend Maven	Compilation et tests ; profil Failsafe exécuté via mvn clean verify -Pfailsafe.
Tests backend	7 tests automatisés exécutés avec succès.
Frontend	Tests Vitest / Vue Test Utils et build Vite validés par la CI.
Conteneur backend	Image construite et démarrée ; API joignable (HTTP 200).
Référentiel CI	GitHub Actions ; runner Ubuntu 24.04.
Déploiement	Pas de déploiement de production démontré. Ne pas assimiler build/CI à une CD complète.

Rapports et collecte de preuves

- **Tests Maven :** target/surefire-reports et target/failsafe-reports lorsqu'ils sont produits.
- **Tests frontend :** logs et résultats des étapes Vitest de la CI.
- **Campagne JMeter :** paramètres de charge, fichier de résultats, distribution des temps de réponse, débit et erreurs.
- **Traçabilité :** conserver l'identifiant du commit Git, la plateforme, la version Java, le scénario et l'endpoint.

Commande de vérification locale

mvn clean verify -Pfailsafe

Cette commande a été exécutée avec succès au cours de la validation du backend. Elle doit être lancée depuis le module qui contient le projet Maven. L'export exact des rapports et leur publication GitHub doivent être vérifiés dans le workflow correspondant avant d'être revendiqués comme systématiques.

10 Limites, risques et décisions

La preuve de concept répond à l'objectif d'apprentissage architectural : établir la faisabilité d'un service de recommandation hospitalière. Les limites suivantes doivent rester explicites devant les parties prenantes.

Sujet	Statut de la preuve	Suite recommandée
Fonctions de recherche	Démontrées sur données simulées.	Sécuriser les critères d'acceptation et fiabiliser les données.
Performance de l'API locale	Validée sur le seul endpoint et run JMeter décrits.	Rejouer sous infrastructure représentative.
Service de routage	Dépendance externe sensible aux quotas 429.	Résilience, monitoring et contractualisation.
Continuité des soins	Non démontrée en conditions réelles.	Tests de panne, redondance et reprise.
Sécurité complète	Aucune authentification / autorisation de production démontrée.	Gestion des secrets, contrôle des accès et HTTPS.
Architecture microservices cible	Le PoC valide un bloc backend et une interface, pas la plateforme complète.	Définir les interfaces d'intégration et la modularisation future.
Événements métier	Bus d'événements non implémenté dans le PoC.	Spécifier publication et réservation d'événement.
KPI hospitaliers	90 % et délai 18,25 → 12 min non mesurés.	Protocole métier avec données opérationnelles autorisées.

Conclusion pour la revue d'architecture

Le PoC dispose d'une base technique opérationnelle : API REST documentée, tests backend et frontend, interface Vue, exécution Docker et benchmark local conforme au seuil considéré. Il ne valide ni l'ensemble de l'architecture cible, ni les engagements de service, ni les KPI cliniques. La poursuite vers l'industrialisation reste conditionnée à la résolution documentée des écarts ci-dessus.

Traçabilité : dépôt source du PoC, workflow GitHub Actions, fichier SQL de test et rapport de performance JMeter.